Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Seguridad de la Información • CE-1115

Taller de Criptografía Aplicada

Chat cifrado entre dos procesos locales con criptografía híbrida

SecureChat: descripción del protocolo

¿Qué viaja por el socket y qué nunca sale de cada extremo?

RSA-OAEP • AES-256-GCM • Handshake • Nonce • Authentication tag

Repositorio: `https://github.com/Adriel23456/SecureChat_Workshop`

PROFESOR

MSc. Andrés F. Vargas Rivera

INTEGRANTES — TRABAJO EN PAREJAS

Nombre	Carné
Adriel S. Chaves Salazar	2021031465
Henry D. Nuñez Pérez	2022089224

II Semestre, 2026 • [Fecha de entrega] • Cartago, Costa Rica

Índice

1 Introducción y objetivo **1**
- 1.1 Alcance . . . 1
- 1.2 Entorno de desarrollo . . . 1

2 Arquitectura general **1**
- 2.1 Organización del código . . . 1
- 2.2 Concurrencia . . . 2

3 Material criptográfico **2**
- 3.1 Par de llaves asimétricas . . . 2
- 3.2 Llave pública y llave privada: qué hace cada una . . . 2
- 3.3 Huella de la llave pública . . . 3
- 3.4 Distribución de las llaves públicas . . . 3
- 3.5 Uso del par de llaves de cada participante . . . 3
- 3.6 Clave de sesión . . . 4
- 3.7 Qué es público, qué es secreto y qué viaja por la red . . . 4

4 Establecimiento de la sesión (handshake) **4**
- 4.1 Mensajes del handshake . . . 5
- 4.2 Confirmación de la sesión . . . 5
- 4.3 Manejo de errores en el handshake . . . 5

5 Protección de los mensajes **5**
- 5.1 AES-256-GCM . . . 6
- 5.2 Generación del nonce . . . 6
- 5.3 Verificación en el receptor . . . 6
- 5.4 Protección contra repetición . . . 6

6 Formato de los mensajes transmitidos **6**

7 Conversación bidireccional y cierre de sesión **7**
- 7.1 Flujo de un mensaje . . . 7
- 7.2 Presentación en consola . . . 7
- 7.3 Cierre controlado . . . 7

8 Detección de manipulación **7**

9 Análisis de seguridad **7**
- 9.1 Modelo de adversario . . . 7
- 9.2 Cumplimiento de los requisitos mínimos . . . 8
- 9.3 Limitaciones conocidas . . . 8

10 Preguntas de cierre **8**

11 Evidencias de la demostración **9**

Referencias **10**

A Ejecución y estructura del repositorio **11**

B Declaración de uso de IA **11**

1. Introducción y objetivo

SecureChat es una aplicación de consola que mantiene una conversación bidireccional entre dos procesos que se ejecutan en la misma computadora, conectados por un socket TCP sobre `localhost`. La criptografía **asimétrica** se usa para establecer la sesión y la criptografía **simétrica autenticada** protege cada mensaje.

[**PENDIENTE:** *Resumen de la solución final en un párrafo: lenguaje, biblioteca, algoritmos, tamaños de llave y resultado de la demostración.*]

1.1. Alcance

[**PENDIENTE:** *Qué cubre y qué no cubre la solución (por ejemplo, autenticación de la llave pública, persistencia, más de dos participantes).*]

1.2. Entorno de desarrollo

Tabla 1.1: Entorno de desarrollo y demostración

Elemento	Detalle
Sistema operativo	Windows [PEND.]
Lenguaje	Python [PEND.]
Biblioteca criptográfica	`cryptography` [PEND.]
Comunicación	Sockets TCP/IP sobre `127.0.0.1`, puerto [PEND.]
Repositorio	`https://github.com/Adriel23456/SecureChat_Workshop`

2. Arquitectura general

El mismo programa se ejecuta dos veces, una instancia por participante (figura 2.1). Bob espera conexiones y Alice inicia la conexión.

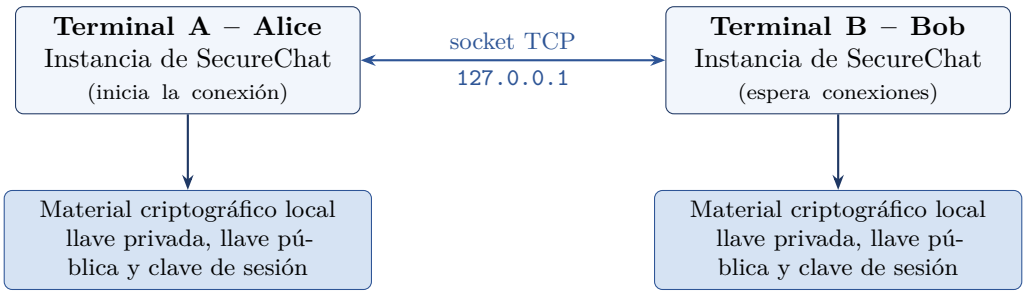


Figura 2.1: Arquitectura de SecureChat: dos instancias del mismo programa conectadas por TCP en la misma computadora.

2.1. Organización del código

[**PENDIENTE:** *Módulos finales y responsabilidad de cada uno (red, framing, llaves, handshake, cifrado, interfaz de consola).*]

2.2. Concurrencia

[**PENDIENTE:** *Cómo se reciben mensajes mientras el usuario escribe (hilo receptor y entrada de teclado) y por qué en Windows no se usa* `select()` *sobre* `stdin`.]

3. Material criptográfico

Cada participante dispone de su propio par de llaves RSA, generado y guardado dentro de su carpeta (`srcAlice/keys/` y `srcBob/keys/`). Ninguna de estas carpetas se versiona: la regla `keys/` del `.gitignore` las excluye del repositorio.

3.1. Par de llaves asimétricas

Tabla 3.1: Parámetros del par de llaves asimétricas

Parámetro	**Valor**
Algoritmo	RSA
Tamaño del módulo	3072 bits (se rechaza al cargar cualquier llave menor a 2048 bits)
Exponente público	$e = 65537$
Generación	`rsa.generate_private_key` de `cryptography`, con el generador aleatorio seguro del sistema operativo
Relleno para cifrar	OAEP con SHA-256 y MGF1(SHA-256) (uso en el handshake, apartado 4)
Relleno para firmar	PSS con SHA-256 y MGF1(SHA-256)
Formato de la llave privada	PEM, PKCS#8, **cifrada** con una frase de paso (`BestAvailableEncryption`)
Formato de la llave pública	PEM, *SubjectPublicKeyInfo* (X.509)
Frase de paso	Mínimo 8 caracteres; se pide sin eco en pantalla (`getpass`) y nunca se guarda
Archivos	`keys/private_key.pem` y `keys/public_key.pem` dentro de la carpeta de cada participante

Se eligió RSA-3072 en lugar del mínimo de 2048 bits porque ofrece alrededor de 128 bits de seguridad, un nivel que se mantiene vigente a largo plazo. El costo es una generación de llaves algo más lenta (uno o dos segundos), que ocurre una sola vez. El exponente $e = 65537$ es el valor estándar: verificar y cifrar con él es rápido y no presenta las debilidades de los exponentes muy pequeños.

3.2. Llave pública y llave privada: qué hace cada una

Las dos llaves están relacionadas matemáticamente: lo que una hace, solo la otra lo deshace. La llave pública (n, e) se deriva de la privada, pero el camino inverso exige factorizar n, lo cual es inviable para un módulo de 3072 bits. Por eso:

- **La llave pública puede compartirse.** Sirve para *cifrar hacia* su dueño y para *verificar* sus firmas. Conocerla no permite descifrar ni firmar.
- **La llave privada debe permanecer bajo control exclusivo de su dueño.** Es la única que puede *descifrar* lo cifrado con la pública y *firmar*. Nunca se muestra en pantalla, nunca se escribe sin cifrar y nunca viaja por el socket.

- **La llave pública no reemplaza a la privada.** Con la pública de Bob, Alice puede proteger la clave de sesión para que solo Bob la lea, pero no puede leerla ella misma una vez cifrada ni hacerse pasar por Bob. Si la pública bastara para descifrar, cualquiera que la conociera (es decir, todos) podría hacerlo.

Prueba de correspondencia. La opción *Verificar mis llaves* del menú carga ambos archivos y ejecuta tres comprobaciones: (1) el módulo n y el exponente e derivados de la privada coinciden con los de la pública; (2) una firma PSS hecha con la privada es aceptada por la pública; y (3) un valor aleatorio cifrado con OAEP usando la pública se recupera con la privada. El valor de prueba es aleatorio, se descarta y solo se informa "aprobado" o "fallido".

3.3. Huella de la llave pública

La huella es el resumen SHA-256 de la codificación DER de la llave pública: 32 bytes que se muestran como `SHA256:ab:cd:...`. Una llave de 3072 bits ocupa cientos de bytes y no se puede comparar a simple vista; la huella sí. Además, si un atacante sustituye la llave pública durante el handshake, la huella cambia por completo.

La huella es información pública: mostrarla no revela ningún secreto. Es lo único de las llaves que el programa muestra en pantalla.

3.4. Distribución de las llaves públicas

Decisión. Se adoptó un modelo mixto. La llave pública viaja por el socket durante el handshake, pero antes de usarla el receptor compara su huella SHA-256 con una huella de confianza registrada previamente. Para registrarla, cada participante muestra su huella en su terminal y el otro la copia por un canal independiente de la red (*fuera de banda*). Si las huellas no coinciden, el handshake se aborta. Así se cierra la puerta al atacante en el medio que sustituye la llave pública, un riesgo que la criptografía por sí sola no resuelve si no se autentica la llave.

Las huellas de confianza se guardan en `srcBob/keys/trusted_alice_fingerprint.txt` y `srcAlice/keys/trusted_bob_fingerprint.txt`.

3.5. Uso del par de llaves de cada participante

- **Par de Bob:** Alice cifra la clave de sesión con la llave pública de Bob (RSA-OAEP) y solo Bob puede recuperarla con su privada. Que Bob logre confirmar la sesión con esa clave demuestra que posee la privada correspondiente a la huella registrada.
- **Par de Alice:** Alice firma su mensaje del handshake con su llave privada (RSA-PSS) y Bob verifica la firma con la llave pública de Alice, previamente autenticada por su huella. Así Bob sabe que la clave de sesión la generó Alice y no un tercero.

Con este esquema la autenticación es **mutua** y cada par de llaves cumple una función concreta. [**PENDIENTE:** *Confirmar con la implementación del Paso 3.*]

3.6. Clave de sesión

[**PENDIENTE:** *Cómo y cuándo se genera la clave de sesión (256 bits, generador aleatorio criptográfico, una nueva por sesión, nunca fija en el código) y cuánto vive. Se completa en el Paso 3.*]

3.7. Qué es público, qué es secreto y qué viaja por la red

Tabla 3.2: Clasificación del material y de los datos del protocolo

Elemento	Dónde vive	¿Viaja por el socket?	Nivel
Llave privada (Alice, Bob)	`keys/private_key.pem`, cifrada; en memoria solo mientras se usa	Nunca	**Secreto**
Frase de paso	Solo en la memoria de su dueño	Nunca	**Secreto**
Llave pública	`keys/public_key.pem`	Sí, en el handshake	**Público**
Huella de la llave pública	Pantalla y archivo de confianza del par	No (se compara fuera de banda)	**Público**
Clave de sesión	Memoria de ambos procesos	Solo cifrada con RSA-OAEP	**Secreto**
Plaintext del usuario	Solo en la terminal local	Nunca (tras el handshake)	**Secreto**
Nonce	Generado por mensaje	Sí, junto al ciphertext	**Público**
Ciphertext	En tránsito	Sí	**Público**
Authentication tag	En tránsito	Sí	**Público**
Metadatos (puertos, tamaños, horas)	Encabezados TCP/IP	Sí	**Público**

4. Establecimiento de la sesión (handshake)

El handshake se completa antes de permitir cualquier mensaje de usuario (figura 4.1).

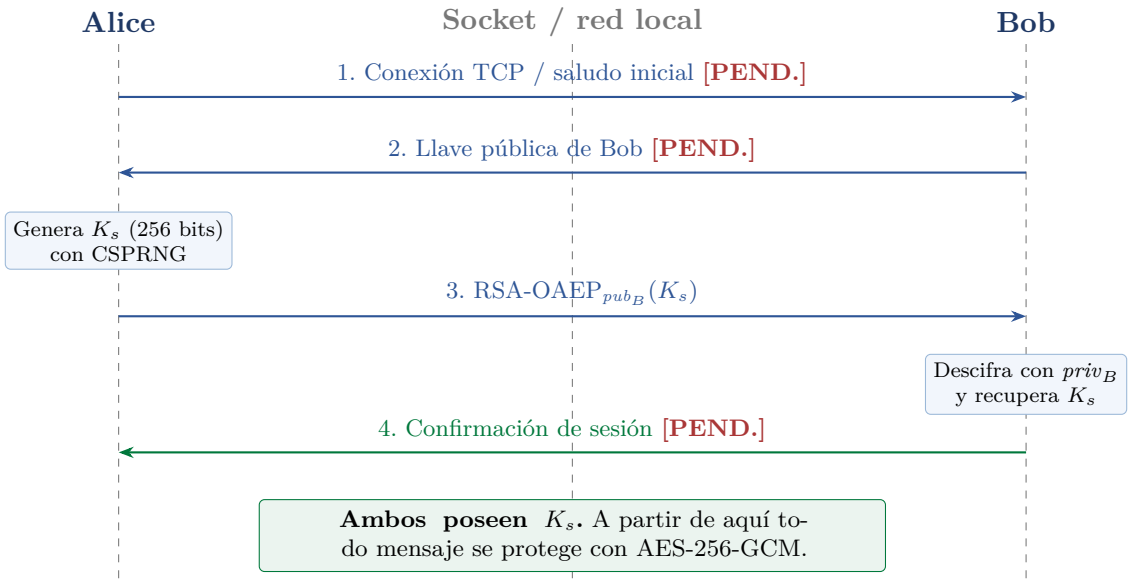


Figura 4.1: Secuencia del handshake (versión preliminar; se actualizará con los mensajes reales).

4.1. Mensajes del handshake

Tabla 4.1: Mensajes del handshake

#	Emisor	Tipo	Contenido y protección
1	Alice	[PEND.]	[PEND.]
2	Bob	[PEND.]	[PEND.]
3	Alice	[PEND.]	[PEND.]
4	Bob	[PEND.]	[PEND.]

4.2. Confirmación de la sesión

[**PENDIENTE:** *Cómo se prueba que ambos extremos tienen la misma clave de sesión y qué se muestra en pantalla al establecerse la sesión segura.*]

4.3. Manejo de errores en el handshake

[**PENDIENTE:** *Qué ocurre si la llave pública es inválida, el descifrado falla, la confirmación no coincide o se agota el tiempo. Nunca se continúa sin cifrado.*]

Decisión pendiente. Autenticación de la llave pública de Bob. Sin verificarla, un atacante activo en el medio podría sustituirla. [**PENDIENTE:** *Decisión adoptada (por ejemplo, verificación de la huella SHA-256 fuera de banda) y justificación.*]

5. Protección de los mensajes

5.1. AES-256-GCM

Tabla 5.1: Parámetros del cifrado simétrico autenticado

Parámetro	Valor
Algoritmo	AES-256-GCM (cifrado autenticado, AEAD)
Tamaño de clave	256 bits
Tamaño del nonce	96 bits
Tamaño del tag	128 bits
Datos asociados (AAD)	**[PEND.]**
Codificación del texto	**[PEND.]**

5.2. Generación del nonce

[**PENDIENTE:** *Estrategia adoptada (contador por dirección o aleatorio), cómo se evita la repetición con la misma clave considerando que ambos sentidos comparten la clave de sesión, y qué pasa si el contador se agota.*]

5.3. Verificación en el receptor

[**PENDIENTE:** *El receptor verifica el tag antes de aceptar el plaintext; si falla, el mensaje se rechaza y no se muestra contenido.*]

5.4. Protección contra repetición

[**PENDIENTE:** *Uso de números de secuencia autenticados o justificación si no se implementa.*]

6. Formato de los mensajes transmitidos

TCP entrega un flujo de bytes sin límites de mensaje, por lo que cada mensaje se delimita con un prefijo de longitud. [**PENDIENTE:** *Confirmar.*]

Tabla 6.1: Estructura de una trama cifrada (propuesta preliminar)

Campo	Tamaño	Descripción
Longitud	**[PEND.]**	Tamaño del resto de la trama
Tipo	**[PEND.]**	Tipo de mensaje
Secuencia	**[PEND.]**	Número de mensaje por dirección
Nonce	12 bytes	Único por cifrado con la misma clave
Ciphertext	variable	Mensaje cifrado
Tag	16 bytes	Authentication tag de GCM

[**PENDIENTE:** *Diagrama de la trama en bytes y ejemplo real capturado.*]

Codificación no es cifrado. La representación en hexadecimal o Base64 se usa solo para mostrar o transportar bytes. No ofrece confidencialidad.

7. Conversación bidireccional y cierre de sesión

7.1. Flujo de un mensaje

Mensaje legible → AES-GCM cifra y autentica → ciphertext en tránsito → verificación y descifrado → mensaje recuperado.

[**PENDIENTE:** *Diagrama de secuencia de un mensaje en cada sentido.*]

7.2. Presentación en consola

Tabla 7.1: Distinción visual en la consola

Etiqueta	Significado
[**PEND.**]	Mensaje local (plaintext antes de cifrar)
[**PEND.**]	Datos cifrados enviados
[**PEND.**]	Datos cifrados recibidos
[**PEND.**]	Mensaje recuperado tras verificar el tag
[**PEND.**]	Error o mensaje rechazado

7.3. Cierre controlado

[**PENDIENTE:** *Comando de salida, mensaje de cierre cifrado y comportamiento del otro extremo.*]

8. Detección de manipulación

[**PENDIENTE:** *Mecanismo de la prueba controlada (alteración en el emisor o proxy intermedio), qué byte se altera y por qué.*]

[**PENDIENTE:** *Resultado: el receptor detecta el tag inválido, rechaza el mensaje y no muestra plaintext. Política tras el rechazo (descartar o cerrar la sesión).*]

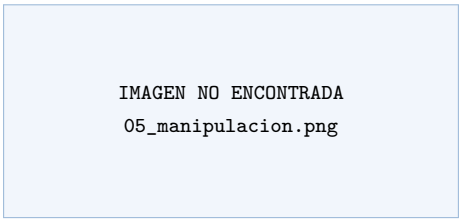


Figura 8.1: Rechazo de un ciphertext alterado.

9. Análisis de seguridad

9.1. Modelo de adversario

[**PENDIENTE:** *Adversario pasivo (escucha el socket) y activo (modifica, repite o inyecta), en la línea del modelo Dolev-Yao visto en clase.*]

9.2. Cumplimiento de los requisitos mínimos

Tabla 9.1: Requisitos mínimos del taller y su cumplimiento

	Requisito	Cómo se cumple	Evidencia
✓	Python 3 y sockets TCP/IP	[PEND.]	[PEND.]
✓	Biblioteca criptográfica reconocida	[PEND.]	[PEND.]
✓	Handshake basado en criptografía asimétrica	[PEND.]	[PEND.]
✓	Clave de sesión dinámica, no fija en el código	[PEND.]	[PEND.]
✓	Ninguna llave privada viaja por el socket	[PEND.]	[PEND.]
✓	Mensajes cifrados con esquema autenticado	[PEND.]	[PEND.]
✓	Nonce correcto y no repetido	[PEND.]	[PEND.]
✓	Rechazo de tags inválidos	[PEND.]	[PEND.]
✓	Evidencia de plaintext, ciphertext enviado y recibido, y plaintext recuperado	[PEND.]	[PEND.]
×	Base64 o hexadecimal no sustituyen al cifrado	[PEND.]	[PEND.]
×	No se usa ECB	[PEND.]	[PEND.]

9.3. Limitaciones conocidas

[**PENDIENTE:** *Por ejemplo: autenticación de la llave pública, ausencia de secreto hacia adelante (forward secrecy) al transportar la clave con RSA, análisis de tráfico (tamaños y tiempos visibles), y la imposibilidad de borrar con certeza secretos de la memoria en Python.*]

10. Preguntas de cierre

1. **¿Qué problema resuelve la criptografía asimétrica durante el handshake?**
   [**PENDIENTE:** *Respuesta.*]
2. **¿Por qué la conversación utiliza criptografía simétrica después del handshake?**
   [**PENDIENTE:** *Respuesta.*]
3. **¿Qué información puede observar un tercero que capture los paquetes de la conversación?**
   [**PENDIENTE:** *Respuesta.*]
4. **¿Por qué el nonce puede viajar junto al ciphertext y por qué no debe repetirse?**
   [**PENDIENTE:** *Respuesta.*]
5. **¿Qué función cumple el authentication tag?**
   [**PENDIENTE:** *Respuesta.*]

6. **¿Qué ocurriría si la clave de sesión se transmitiera sin protección?**
   [**PENDIENTE:** *Respuesta.*]
7. **¿Qué ocurriría si un atacante obtiene una de las claves privadas?**
   [**PENDIENTE:** *Respuesta.*]

11. Evidencias de la demostración

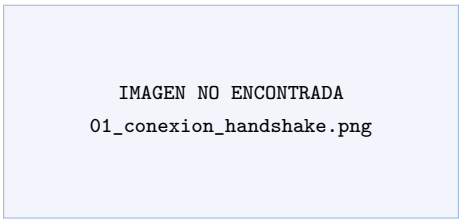


Figura 11.1: Dos instancias en ejecución y handshake completado.

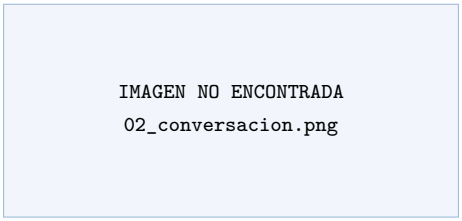


Figura 11.2: Conversación con al menos tres mensajes por sentido.

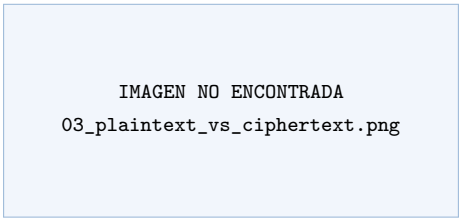


Figura 11.3: Comparación entre el mensaje legible y el ciphertext transmitido.

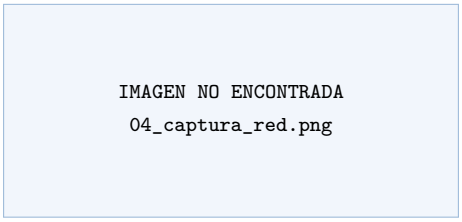


Figura 11.4: (Opcional) Captura del tráfico de `localhost`.

[**PENDIENTE:** *Enlace al video de demostración.*]

Referencias

A. Ejecución y estructura del repositorio

[**PENDIENTE:** *Estructura final del repositorio y comandos para instalar, generar llaves e iniciar cada instancia en Windows (resumen del README).*]

B. Declaración de uso de IA

[**PENDIENTE:** *Herramientas usadas, para qué y cómo se verificó el resultado.*]